

Multi-Objective Structured Pruning of LLMs for Latency and Model Size Optimization

Muhammad Junaid Ali

Inria, University of Lille

Lille, France

Muhammad-junaid.ali@inria.fr

0000-0003-0208-9419

Smail Niar

LAMIH UMR CNRS 8201, UPHF

INSA Hauts-de-France

Valenciennes, France

smail.niar@uphf.fr

0000-0002-7550-484X

El Ghazali Talbi

University of Lille

CNRS, Inria, Centrale Lille

Lille, France

el-ghazali.talbi@univ-lille.fr

0000-0003-4549-1010

Abstract—Large Language Models (LLMs) have achieved widespread adoption due to their strong reasoning and query-response capabilities. However, deploying them in embedded and edge computing environments remains challenging because strict latency, memory, and resource constraints limit efficient execution. Model pruning is a practical solution for reducing complexity while preserving performance, but existing approaches mainly emphasize accuracy preservation or parameter reduction, treating latency mostly as a post-pruning metric. Moreover, jointly pruning layers, attention heads, and Multi Layer Perception (MLP) layers creates a large design space where sparsity allocations yield different accuracy-latency trade-offs. To address these limitations, we propose a hardware-aware multi-objective structured pruning framework for LLM compression. In the first stage, the framework searches for a Pareto set of coarse structured pruning configurations by removing latency-costly sub-components while preserving performance. Rather than relying on a single Pareto candidate, the second stage applies Parallel Bayesian Optimization (PBO) over the returned Pareto-front solutions to allocate layer-wise sparsity ratios under latency constraints, enabling broader exploration of latency-aware configurations. We benchmark the proposed method across multiple LLMs and sparsity ratios against existing pruning and sparsity allocation methods. Experimental results across four LLM families at 37.5% and 50% sparsity show that the proposed approach outperforms existing methods on downstream commonsense reasoning tasks on average, achieves lower perplexity on WikiText-2, C4, and FineWeb-Edu, and reduces 100-token inference latency to 73.8512s on an NVIDIA A100 GPU and 843s on the evaluated Jetson Nano edge platform.

Index Terms—large language models, structured pruning, multi-objective optimization, edge deployment

I. INTRODUCTION

Language models are not new, but transformer architectures have greatly accelerated the development of Large Language Models (LLMs). These models are designed to understand, generate, and predict human language, and they are now used in tasks such as machine translation, natural language generation, speech recognition, and optical character recognition. Some famous chatbots include ChatGPT, Claude, and Gemini. However, their large model size creates a major deployment challenge, especially for small edge devices or local environments with limited memory and strict latency requirements.

Two primary approaches exist for obtaining small language models. The first is to train a compact architecture from scratch, but this often requires substantial data and computation. Alternatively, researchers can compress existing large models

to preserve performance at a smaller scale. Existing studies have shown that LLMs contain substantial redundancy at different structural levels [1], [2], [4], [6]. In transformer-based LLMs, each block mainly consists of a Multi-Head Attention (MHA) module and a feed-forward Multi-Layer Perceptron (MLP) module. Since these modules differ in their impact on accuracy, parameter count, and latency, pruning can be applied at multiple granularities, from entire transformer blocks to individual attention heads or MLP neurons.

Pruning approaches are typically divided into unstructured, semi-structured, and structured methods. Unstructured pruning removes individual weights and can yield high sparsity, but its irregular patterns often need specialized hardware or sparse kernels to improve speed. Structured pruning instead eliminates whole units, such as layers, attention heads, channels, or MLP neurons, creating dense smaller networks that run more efficiently on standard edge devices.

Table I compares the proposed framework with structured LLM pruning methods across key design aspects. The comparison highlights that multi-objective search, sparsity allocation, and edge evaluation are not jointly considered in existing studies.

Despite the effectiveness of existing structured pruning methods, two challenges remain. The most relevant prior works include LLM-Pruner [1], SliceGPT [4], ShortGPT [6], BlockPruner [5], 2SSP [3], and CFSP [15]. First, depth-wise pruning must preserve model accuracy while reducing deployment costs, such as latency and model size. Second, after a coarse architecture is selected, the remaining pruning budget must be allocated across layers and components rather than applied uniformly. In this work, we address these issues through a two-stage framework:

The motivation behind the two-stage design is to reduce the complexity of the pruning search. Searching layers, attention heads, and MLP dimensions simultaneously creates a very large design space, which can lead to underexploration and make it difficult to identify effective configurations.

Therefore, Stage 1 performs coarse-grained Pareto search to remove MHA and MLP sub-blocks with lower performance contribution and higher latency or model-size cost. Stage 2 then refines each retained architecture through layer-wise sparsity

Method	Layer/Block Pruning	Attention/MHA Pruning	MLP/FFN Pruning	Two-Stage Design	Multi-Objective Search	Sparsity Allocation	Edge Evaluation
ShortGPT [6]	✓	–	–	–	–	–	–
SliceGPT [4]	–	✓	✓	–	–	–	–
BlockPruner [5]	✓	✓	✓	–	–	–	–
2SSP [3]	–	✓	✓	✓	–	–	–
CFSP [15]	✓	✓	✓	✓	–	–	–
Proposed	✓	✓	✓	✓	✓	✓	✓

TABLE I

COMPARISON OF THE PROPOSED STRUCTURED LLM PRUNING APPROACH WITH EXISTING METHODS ACROSS KEY DESIGN ASPECTS.

allocation and sub-component pruning. This decomposition makes the search more efficient and stable, while the ablation studies evaluate whether the two-stage strategy achieves lower latency than using either stage independently.

The contributions of this paper are as follows:

- We propose a two-stage, multi-objective structured pruning framework for LLM compression. The first stage identifies a Pareto set of layer-wise pruning configurations by jointly optimizing performance preservation and latency reduction. Instead of relying on a single Pareto candidate from the first stage, the second stage applies parallel Bayesian optimization over the returned Pareto-front solutions to identify a pruning configuration that maximizes performance retention while minimizing latency.
- We propose a parallel Bayesian optimization strategy for layer-wise sparsity allocation, searching for pruning configurations that reduce inference latency while maintaining model performance. In contrast to prior methods that mainly emphasize accuracy preservation, our approach explicitly incorporates latency into the pruning search objective.
- We benchmark the proposed method across multiple LLMs under different sparsity ratios. The results show that the proposed approach improves performance on downstream commonsense reasoning benchmarks under the evaluated sparsity budgets, achieves lower perplexity across multiple language-modeling evaluation datasets, and reduces inference latency while increasing throughput on both a high-performance GPU and an NVIDIA Jetson Nano edge device.

The remainder of this paper is structured as follows: Section II reviews related work, Section III presents the proposed methodology, Section IV describes the experimental settings, Section V reports the results and discussion, and Section VI concludes the paper.

II. RELATED WORK

Pruning methods are commonly grouped into structured, semi-structured, and unstructured approaches.

Early structured pruning approaches include LLM Pruner [1], a task-agnostic method that removes non-essential coupled structures (dependency groups) based on gradient information, recovering performance through Low-Rank Adaptation (LoRA) fine-tuning. Similarly, Block Pruner performs fine-grained structured pruning within transformer blocks by pruning both MLP and MHA sub-components based on component-level importance [5]. Other methods focus on pruning model width and individual columns [1], [3].

SliceGPT uses Principal Component Analysis (PCA) to remove rows and columns across matrices [4]. However,

such approaches often require additional parameters or rigid structural adjustments to maintain compatibility. 2SSP [3] first prunes FFN neurons by output magnitude, then iteratively removes attention sub-modules, **but this staged sequence may miss the global optimum**. Similarly, Blockpruner [5] scores component importance to prune attention heads and MLPs. Unlike 2SSP, it applies block-level importance to both MLP and MHA pruning, but ignores sub-component-level pruning importance [5].

Recently, Coarse-to-Fine Structured Pruning (CFSP) [15] uses inter- and intra-block activation information to guide structured pruning. However, CFSP mainly relies on activation-based coarse-to-fine importance and does not explicitly optimize latency or model size as search objectives across multiple Pareto candidates. Existing layer-wise sparsity allocation methods mainly determine pruning ratios from layer importance, outlier statistics, reconstruction error, or theoretical error-propagation criteria [19]–[23]. Although some of these works report inference speedup after pruning, latency is generally treated as an evaluation metric rather than an explicit search objective. In contrast, our approach uses deployment-oriented objectives during pruning and refines multiple coarse Pareto candidates instead of relying on a single architecture.

Unstructured pruning removes individual weights and can preserve accuracy at high sparsity, but often yields limited inference speedups because standard GPU kernels poorly support irregular sparsity [2].

Although the aforementioned approaches are effective, they often do not explicitly optimize performance–efficiency trade-offs in a hierarchical manner. To address this, we separate global architecture search from local refinement, enabling more balanced optimization of model quality, size, and latency.

III. PROPOSED METHODOLOGY

Our proposed methodology uses a global-to-local pruning framework that separates macro-level architecture search from fine-grained structural refinement. Figure 1 provides an overview of the methodology, and Algorithm 1 summarizes the overall procedure. The example illustrates how Stage 1 produces coarse Pareto candidates and Stage 2 refines them through latency-aware layer-wise allocation and sub-component pruning.

Stage 1 prunes complete MHA and MLP sub-blocks to obtain coarse Pareto-optimal architecture candidates, while Stage 2 uses parallel Bayesian optimization to allocate the remaining sparsity budget and applies importance scoring to prune low-ranked sub-components.

An overview of the proposed pruning framework illustrating the two-stage process

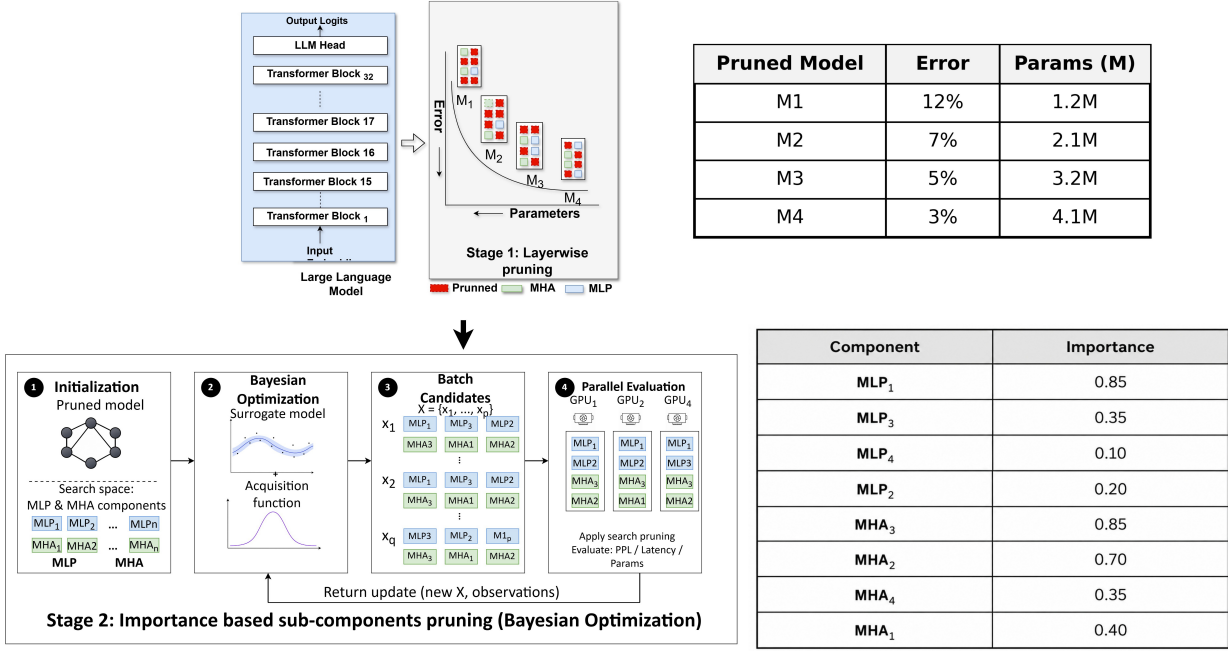


Fig. 1. Overview of the proposed multi-objective structured pruning framework consisting of two different stages.

We next define the MHA and MLP notation used in the pruning formulation.

A. Multi-Head Attention

Multi-Head Attention (MHA) is the sequence-context modeling module of a transformer block. It projects the input sequence into query, key, and value representations and allows each token representation to aggregate information from other token positions through multiple attention heads. Let $X \in \mathbb{R}^{S \times d_{\text{model}}}$ denote the input hidden states, where S is the sequence length and d_{model} is the hidden dimension. For head $h \in \{1, \dots, H\}$, the projections are defined as:

$$Q_h = XW_h^Q, \quad K_h = XW_h^K, \quad V_h = XW_h^V, \quad (1)$$

where $W_h^Q, W_h^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $W_h^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ are head-specific projection matrices, and d_k, d_v denote the query/key and value dimensions per head, respectively. The output of each head is computed as:

$$\text{head}_h = \text{softmax}\left(\frac{Q_h K_h^T}{\sqrt{d_k}}\right) V_h. \quad (2)$$

The outputs of all H heads are concatenated and passed through the output projection matrix $W^O \in \mathbb{R}^{(H \cdot d_v) \times d_{\text{model}}}$:

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) W^O. \quad (3)$$

In Stage 2, all importance estimators are applied under the same structured pruning unit. For MHA, the unit is an attention-head group. For layer l and head h , we define this structurally coupled group as:

$$G_{l,h}^{\text{MHA}} = \{W_{l,h}^Q, W_{l,h}^K, W_{l,h}^V, W_{l,\mathcal{I}_h,:}^O\}, \quad (4)$$

where $\mathcal{I}_h$ denotes the index set of the hidden dimensions associated with head h in the concatenated tensor, and $W_{l,\mathcal{I}_h,:}^O$ denotes the corresponding incoming row slice of the output-projection matrix. The importance estimators used in Stage

Algorithm 1 Two-Stage Pruning with Bayesian Layer-Wise Allocation

- 1: **Input:** Model $\mathcal{M}$, calibration data $\mathcal{D}$, validation data $\mathcal{D}_{\text{val}}$, sparsity budget S
- 2: **Output:** Best pruned model M_{best}
- Stage 1: Multi-Objective Depth-Wise Pruning**
- 3: $\mathcal{C} \leftarrow \text{INITPOPULATION}()$
- 4: **while** stopping criterion is not met **do**
- 5: **for** each candidate $c \in \mathcal{C}$ **do**
- 6: $M_c \leftarrow \text{PRUNESUBBLOCKS}(\mathcal{M}, c)$
- 7: Evaluate M_c using quality loss and latency
- 8: **end for**
- 9: $\mathcal{P} \leftarrow \text{PARETOFRONT}(\mathcal{C})$
- 10: $\mathcal{C} \leftarrow \text{EVOLVE}(\mathcal{P})$
- 11: **end while**
- 12: $\mathcal{C}_{\text{sel}} \leftarrow \text{SELECTPARETOCANDIDATES}(\mathcal{P})$
- Stage 2: Bayesian Allocation and Importance Benchmarking**
- 13: $\mathcal{A} \leftarrow \text{INITBAYESIANOPTIMIZER}(S)$
- 14: $\mathcal{R} \leftarrow \emptyset$ ▷ Evaluated configurations
- 15: **for** each coarse candidate $M_c \in \mathcal{C}_{\text{sel}}$ **do**
- 16: **while** Bayesian optimization budget is not exhausted **do**
- 17: Propose layer-wise allocations $\{a_1, \dots, a_m\}$
- 18: **for** each allocation a_j **in parallel do**
- 19: **for** each importance method g **do**
- 20: Rank subcomponents using g and $\mathcal{D}$
- 21: $M_{c,j,g} \leftarrow \text{PRUNEBYALLOCATION}(M_c, a_j, g)$
- 22: Evaluate the perplexity of $M_{c,j,g}$ on $\mathcal{D}_{\text{val}}$ and measure its latency
- 23: Add $M_{c,j,g}$ and its results to $\mathcal{R}$
- 24: **end for**
- 25: **end for**
- 26: Update $\mathcal{A}$ using the observed perplexity and latency values
- 27: **end while**
- 28: **end for**
- 29: $M_{\text{best}} \leftarrow \text{SELECTBESTCONFIGURATION}(\mathcal{R})$ ▷ Lowest perplexity and latency
- 30: **return** M_{best}

2 assign a score to each group $G_{l,h}^{\text{MHA}}$, and the lowest-scoring heads are removed according to the layer-wise pruning ratios proposed by Bayesian optimization. This group-wise definition allows weight-level estimators, such as Wanda-SP or gradient-based criteria, to be used consistently for structured head pruning by aggregating their scores over all parameters belonging to the same head.

B. Feed-Forward Network

The Feed-Forward Network (FFN), also referred to as the MLP module, applies token-wise nonlinear transformations after attention. Modern LLMs commonly use a gated MLP, such as SwiGLU, consisting of up, gate, and down projections. For an input hidden state $X \in \mathbb{R}^{S \times d_{\text{model}}}$, the MLP computation is formulated as:

$$\text{MLP}(X) = (\text{SiLU}(XW_{\text{Gate}}) \odot XW_{\text{Up}}) W_{\text{Down}}, \quad (5)$$

where $W_{\text{Gate}}, W_{\text{Up}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ffn}}}$ expand the hidden representation to the intermediate dimension d_{ffn} , $W_{\text{Down}} \in \mathbb{R}^{d_{\text{ffn}} \times d_{\text{model}}}$ projects it back to the model dimension, and $\odot$ denotes element-wise multiplication.

In Stage 2, MLP pruning removes structured intermediate-neuron groups. For layer l and intermediate neuron $u \in \{1, \dots, d_{\text{ffn}}\}$, the pruning group is defined as:

$$G_{l,u}^{\text{MLP}} = \{W_{l,u,:}^{\text{Gate}}, W_{l,u,:}^{\text{Up}}, W_{l,u,:}^{\text{Down}}\}, \quad (6)$$

where $W_{l,u,:}^{\text{Gate}}$ and $W_{l,u,:}^{\text{Up}}$ denote the u -th column vectors of the gate and up projections, and $W_{l,u,:}^{\text{Down}}$ represents the u -th row vector of the down projection. Thus, pruning neuron u consistently removes the coupled gate, up, and down projection components associated with that neuron.

C. Stage 1: Multi-Objective Layer-Wise Pruning

Given an LLM $\mathcal{M}$ consisting of $\mathcal{L}$ layers, the objective is to identify the structural sub-blocks, specifically the Multi-Head Attention (MHA) and Feed-Forward Network (FFN) modules, to be pruned. Let $l \in \{1, 2, \dots, \mathcal{L}\}$ denote the layer index, and let $x_{l,\text{MHA}}, x_{l,\text{MLP}} \in \{0, 1\}$ be binary decision variables indicating whether the respective MHA and FFN modules are retained (1) or pruned (0) in layer l . The decision space Θ encompasses all possible pruning configurations across the entire model architecture. For each layer l , the decision space consists of two binary variables, yielding the search space:

$$\Theta = \prod_{l=1}^{\mathcal{L}} \{0, 1\}^2 = \{0, 1\}^{2\mathcal{L}}. \quad (7)$$

Each configuration vector $x \in \Theta$ uniquely maps a specific pruning state across all $\mathcal{L}$ layers.

The structural search is formulated as a multi-objective optimization problem aimed at finding the decision vector $x = \{x_{1,\text{MHA}}, x_{1,\text{MLP}}, \dots, x_{\mathcal{L},\text{MHA}}, x_{\mathcal{L},\text{MLP}}\} \in \Theta$ that simultaneously minimizes performance degradation and structural complexity:

$$\min_{x \in \Theta} (\mathcal{O}_1(x), \mathcal{O}_2(x)), \quad (8)$$

where $\mathcal{O}_1(x)$ denotes the performance loss between the original and pruned models, and $\mathcal{O}_2(x)$ represents the total number of remaining parameters.

To quantify the performance loss $\mathcal{O}_1(x)$, we measure the Kullback-Leibler (KL) divergence between the output distributions of the original model $\mathcal{M}_{\text{original}}$ and the pruned model $\mathcal{M}_{\text{pruned}}$. The KL divergence over the vocabulary token space $\mathcal{V}$ is formulated as:

$$D_{\text{KL}}(P \parallel Q) = \sum_{v \in \mathcal{V}} P(v) \log \left(\frac{P(v)}{Q(v)} \right), \quad (9)$$

where $P(v)$ and $Q(v)$ express the output probability distributions of $\mathcal{M}_{\text{original}}$ and $\mathcal{M}_{\text{pruned}}$, respectively, for a given context.

Layer pruning in Stage 1 is executed by replacing the forward pass of targeted sub-blocks with an identity mapping. By utilizing the identity mapping pathways within the residual blocks, the pruned layers are effectively bypassed, allowing the hidden states to propagate through the network via the skip connections without further computation, as illustrated in Figure 2.

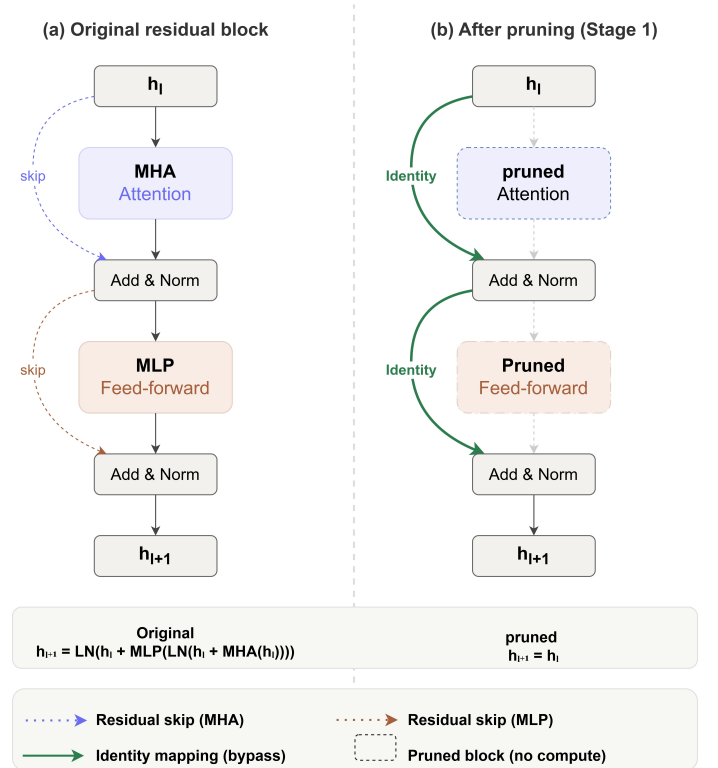


Fig. 2. Block-level pruning in LLMs. (a) Standard transformer block with MHA and MLP. (b) Pruned block where computation is skipped and replaced with an identity connection.

D. Stage 2: Parallel Bayesian Layer-Wise Allocation

Stage 2 takes the Pareto-front candidates returned by Stage 1 and further optimizes the retained architectures. It searches over the remaining layer-wise pruning budgets of these candidates and evaluates the resulting models on Wikitext-2. When multiple candidates exhibit similar validation quality, the lower-cost candidate is selected according to active parameter count or measured latency.

The motivation behind using Bayesian optimization in Stage 2 is that the best layer-wise sparsity allocation is not known in advance. Fixed or uniform sparsity ratios may be suboptimal

because different layers and sub-components contribute differently to accuracy, latency, and parameter reduction. Bayesian optimization guides the search toward promising allocations under latency and performance constraints, while the parallel setting evaluates multiple candidate allocations per iteration to reduce sequential evaluation bottlenecks.

Let $a = \{a_l^{\text{MHA}}, a_l^{\text{MLP}}\}_{l=1}^L$ denote a candidate allocation vector, where $0 \leq a_l^{\text{MHA}}, a_l^{\text{MLP}} \leq 1$ are the pruning ratios assigned to MHA heads and MLP neurons in layer l . The allocation search jointly reduces validation perplexity and latency:

$$f(a) = (\text{PPL}_{\text{val}}(M_a), T(M_a)), \quad M_a = \text{Prune}(M^{(1)}, a, g). \quad (10)$$

The feasible allocation set is defined by the remaining pruning budget:

$$\mathcal{A}_{\text{feas}} = \{a \in \mathcal{A} : |\mathcal{B}(a) - S_{\text{rem}}| \leq \epsilon\}, \quad (11)$$

where candidate allocations are ranked according to their validation perplexity and latency scores. Let

$$\tau = (\tau_1, \dots, \tau_{|\mathcal{A}_{\text{feas}}|})$$

denote the permutation of feasible-candidate indices obtained by sorting the allocations in ascending lexicographic order of validation perplexity and latency:

$$\tau = \text{argsort}_{i \in \{1, \dots, |\mathcal{A}_{\text{feas}}|\}} (\text{PPL}_{\text{val}}(M_{a_i}), T(M_{a_i})). \quad (12)$$

The optimal allocation is then selected as the highest-ranked feasible candidate:

$$a^* = a_{\tau_1}. \quad (13)$$

Thus, candidates with lower validation perplexity are preferred with lower latency. Here, $M^{(1)}$ denotes a Stage-1 Pareto candidate, g denotes an importance strategy, $\text{PPL}_{\text{val}}(\cdot)$ denotes validation perplexity, and $T(\cdot)$ denotes measured latency. Furthermore, S_{rem} is the remaining pruning budget, while ϵ accounts for rounding continuous pruning ratios to feasible structured units, such as integer attention heads and MLP neurons. The budget function $\mathcal{B}(a)$ maps the layer-wise pruning ratios to the corresponding fraction of removed parameters. For each allocation, the importance strategy ranks the sub-components within the same Bayesian allocation framework. This procedure preserves the multi-objective nature of the search while providing a deterministic, deployment-oriented selection criterion.

IV. EXPERIMENTAL SETTINGS

All experiments were conducted on three NVIDIA A100 80GB GPUs using PyTorch 2.1 and the Hugging Face Transformers library. We used the lm-evaluation-harness library for downstream task evaluation. Evaluations were performed in a zero-shot setting on Massive Multitask Language Understanding (MMLU), HellaSwag (HS), AI2 Reasoning Challenge (ARC)-Easy/ARC-Challenge, Physical Interaction Question Answering (PIQA), and WinoGrande (WG). We report average accuracy on these benchmarks. For edge benchmarking, we used the evaluated NVIDIA Jetson platform and report its measured latency separately from the A100 results to avoid conflating desktop-GPU and edge-device execution behavior.

In the first stage, we implemented the Non-dominated Sorting Genetic Algorithm II (NSGA-II) via Pymoo with simulated

binary crossover and mask mutation (population size 40, 40 generations), and used 1024 random FineWeb-Edu samples to compute Kullback–Leibler (KL) divergence. The choice of NSGA-II is motivated by its effectiveness in solving multi-objective optimization problems [14]. For the implementation of BO, GPyTorch and BoTorch are used.

Across four models, we run each search independently for each target sparsity rate, 37.5% and 50%. Stage 1 uses NSGA-II with a population of 40 for 40 generations, giving 1,600 evaluations per sparsity rate and 3,200 in total. It takes about 4 hours per sparsity rate. Stage 1 returns 8–10 candidates on average, so 18 architectures are evaluated downstream. Stage 2 uses 20 warm-up and 20 BO trials per candidate, giving 40 evaluations per candidate and 720 Stage-2 evaluations in total. It takes about 4 hours per sparsity rate. With two GPUs, this corresponds to about 14 GPU-hours per sparsity rate and 28 GPU-hours for both. Existing studies are mostly knowledge-driven and usually take 2–3 hours, while optimization-based methods such as EvoPress take about 48 hours. However, EvoPress targets unstructured sparsity and does not consider latency or layer-wise sparsity, which our optimization approach benefits from.

We define the pruning ratio $S \in [0, 1]$ as the target sparsity fraction corresponding to $100S\%$ pruning of the structured components or parameters considered in the search, following standard practices in existing studies [3], [6]. The distribution of this pruning budget between Stage 1 and Stage 2 is determined by the Stage-1 Pareto solutions, which may remove different fractions of the model.

In Stage 1, pruning is applied up to the target budget S , generating multiple feasible coarse-pruned solutions. Each solution i is characterized by its achieved sparsity fraction p_i , defined as:

$$p_i = \frac{\text{Pruned parameters}_i}{\text{Total parameters}} \quad (14)$$

Stage 2 then allocates the remaining pruning budget, $S_{\text{rem}} = S - p_i$, for each respective solution using parallel Bayesian optimization. The optimized allocation specifies how much additional sub-component pruning should be applied within the retained layers to obtain a lower-latency configuration while satisfying the overall sparsity target.

We evaluated the proposed method at 37.5% and 50% pruning rates across four LLM families to assess performance under different sparsity levels and model architectures. Comparisons with existing studies were conducted without LoRA recovery fine-tuning, highlighting the effectiveness of the pruning strategy itself. Since some baselines may rely on different implementations or recovery procedures, these results are interpreted as controlled pruning comparisons under the reported evaluation protocol rather than exhaustive baseline re-implementations.

V. RESULTS AND DISCUSSION

Experimental Methodology

To evaluate the effectiveness of the proposed approach we have performed multiple experiments on two different evaluation criteria accuracy on downstream tasks and perplexity

on language-modeling tasks. We compare our approach with multiple existing baseline methods.

Given that our proposed approach utilizes both depth-wise and component-wise pruning, we benchmark against the structural pruning baselines reported in Table II, including ShortGPT [6], Sliding Window [7], Block Pruner [5], EvoPress [10], SliceGPT [4], 2SSP [3] and CFSP [15], a coarse-to-fine pruning approach. We compare against CFSP only on LLaMA-2-7B, as their method supports only Llama-based architectures.

We evaluated our approach using both the WikiText-2 and Mixed Calibration datasets as validation sets. Due to time and resource constraints, we conducted five independent runs for the proposed method and reported their average and standard deviation; baseline values are reported under the same table for comparison.

For perplexity and downstream tasks, we perform experiments at sparsity rates of 37.5% and 50%. Following recent LLM compression studies that use similar pruning budgets [3], [4], this allows us to examine whether moderate and aggressive pruning preserve general reasoning performance and language-modeling quality as the pruning regime becomes more challenging.

Experimental Results

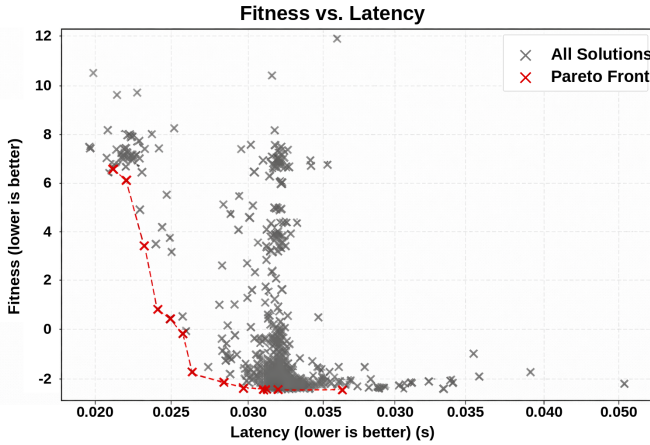


Fig. 3. Visualization of the Pareto front and non-Pareto solutions at a sparsity rate of 37.5%.

Figure 3 shows the visualization of solutions obtained after the multi-objective search on 37.5% pruning ratio, including individual solutions and Pareto front solutions. The accuracy on downstream tasks is reported for four popular models: LLaMA 2-7B [12], Qwen 2.5 7B [16], Phi-3 14B [17] and Mistral-v0.3-7B [13]. The detailed results are provided in Table II.

The results at 37.5% and 50% sparsity show that the proposed approach achieves the strongest overall average downstream accuracy across all four model families. In particular, it consistently preserves the aggregate downstream performance under structured pruning, indicating that the compressed models retain a robust accuracy profile across the evaluated setting. At 50% sparsity, competing approaches degrade more sharply, whereas the proposed approach still retains the highest average accuracy across all models.

At 50% sparsity, the method remains especially strong in terms of average downstream accuracy across the evaluated

model families. The results of mixed calibration sets are reported as *Proposed+Mix*. Figure 4 visualizes the layer-wise MHA and MLP pruning patterns for the 37.5% sparsity setting. The pruning distribution is non-uniform across layers and components, demonstrating that BO does not apply a fixed pruning ratio universally. Instead, it removes more MHA or MLP capacity from layers that are less critical to the selected objectives, while preserving components that are highly sensitive to average accuracy drops. This behavior explains why the proposed method can reduce latency and active model size without causing the sharp downstream performance degradation typically observed in uniform or single-stage pruning strategies.

The results shows that the proposed approach is not only calibration dependent and perform well in both mixed calibration and wikitext-2.

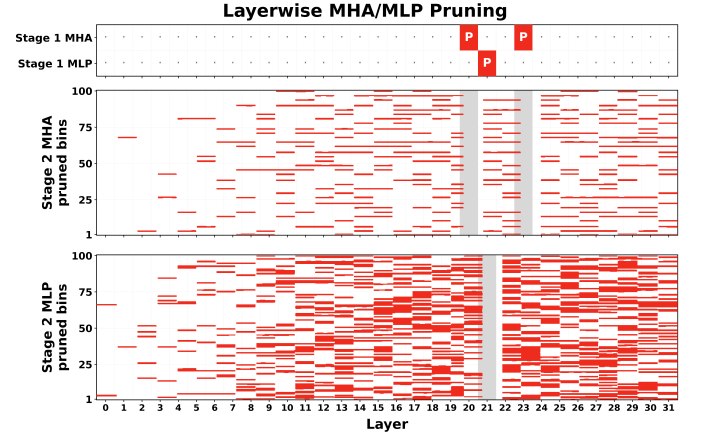


Fig. 4. Layer-wise MHA and MLP pruning patterns under the two-stage strategy at 37.5% sparsity.

In addition to zero-shot accuracy, Table III reports end-to-end latency on 100-token sequences averaged over 10 runs on an NVIDIA A100 and the evaluated NVIDIA Jetson edge platform. These latency values correspond to the full 100-token generation protocol and should not be directly compared with the shorter validation-time latency values reported in the ablation studies. The proposed approach achieves the lowest latency on both platforms (73.8512 s on A100 and 843 s on the Jetson platform), confirming that the multi-objective search and parallel BO search improve deployment efficiency together with accuracy.

Although the proposed approach maintains an overall balance across all evaluated metrics, certain performance gaps arise because Stage-1 prioritizes global latency and efficiency, while Stage-2 relies on task-agnostic importance scores from a general calibration set rather than optimizing for individual benchmarks. Consequently, it selectively prunes components that offer minimal performance contributions while incurring high computational costs. Despite these localized trade-offs, the proposed approach achieves the highest average performance alongside the lowest latency across the tested hardware.

In addition to benchmarking downstream tasks, we computed perplexity (PPL) scores on three benchmarks: WikiText-2, FineWeb-Edu, and C4. The bar chart in Figure 5 compares these scores at two sparsity rates (37.5% and 50%) for two

TABLE II

AVERAGE ZERO-SHOT BENCHMARK RESULTS FOR FOUR LLMs AT 37.5% AND 50% SPARSITY LEVELS. GOLD AND SILVER SHADING DENOTE THE BEST AND SECOND-BEST PERFORMING PRUNED RESULTS, RESPECTIVELY. FOR THE PROPOSED APPROACH, WE REPORT THE MEAN AND STANDARD DEVIATION ACROSS FIVE INDEPENDENT RUNS USING FINEWEB-EDU AND MIXED CALIBRATION SUBSETS.

Method	Mistral-v0.3		LLaMA-2 7B		Qwen-2.5		Phi-3 14B	
	37.5%	50%	37.5%	50%	37.5%	50%	37.5%	50%
<i>Dense</i>	67.08	67.08	60.71	60.71	68.72	68.72	72.19	72.19
ShortGPT [6]	38.13	36.83	42.63	36.89	41.28	32.33	48.34	34.83
Sliding Window [7]	38.79	33.65	41.69	34.96	42.20	31.36	47.80	32.97
Block Pruner [5]	42.14	37.16	42.96	37.19	44.78	35.14	48.16	40.61
EvoPress [10]	44.48	37.51	43.76	38.53	45.09	36.22	48.48	38.46
SliceGPT [4]	41.90	36.34	44.83	37.72	41.56	35.51	48.57	39.23
2SSP [3]	45.73	39.27	47.60	37.76	46.37	41.47	49.10	43.17
CFSP [15]	—	—	49.62	36.94	—	—	—	—
Proposed	49.15±0.24	39.69±0.54	49.79±0.65	38.65±0.63	48.91±0.74	43.39±0.19	49.45±0.35	44.64±0.47
Proposed+Mix	51.92±0.73	46.03±0.69	52.99±0.17	40.39±0.36	51.84±0.29	45.34±0.41	50.09±0.83	45.96±0.85

TABLE III

COMPARISON OF LATENCY BETWEEN THE PROPOSED APPROACH AND EXISTING STUDIES. RESULTS ARE SHOWN IN SECONDS, COMPUTED ON 100-TOKEN TEXT SEQUENCES OVER 10 ITERATIONS ON AN NVIDIA A100 GPU AND THE EVALUATED NVIDIA JETSON EDGE PLATFORM.

Method	NVIDIA A100	NVIDIA Jetson Nano
Sliding Window [7]	82.24162	956
2SSP [3]	92.61473	1150
Block Pruner [5]	85.46769	1074
EvoPress [10]	85.57885	1187
ShortGPT [6]	81.93638	1283
CFSP [15]	76.4123	937
Proposed	73.8512	843

LLMs, LLaMA-2-7B and Mistral-v3. The sub-figures show that the proposed approach generally achieves lower perplexity across the evaluated benchmarks and models, confirming its effectiveness beyond downstream-task accuracy. These results also complement the downstream accuracy and latency tables, where multiple runs are reported to reduce the influence of run-to-run variation and provide a more reliable comparison across pruning configurations.

To study the impact of the sparsity rate (S) on performance, we calculate the perplexity scores on WikiText-2 at different sparsity rates after applying Stage 1 and Stage 2 and compare them with baseline studies, as shown in Figure 10. The results show that varying S significantly impacts performance; while smaller sparsity rates result in only a minor increase in perplexity, a substantial increase is observed beyond 40% sparsity, with the rise becoming more pronounced between 50% and 70%.

This is due to the removal of essential weights that encode the model’s fundamental linguistic patterns, which cannot be recovered once the redundancy threshold is surpassed. The baseline methods did not show increased perplexity until 40%, but after 40% they exhibit a significant increase; however, the proposed approach maintains a lower increase in perplexity compared to existing studies even after 40%. This demonstrates that the proposed approach remains robust even as the sparsity rate increases.

1) *Critical Difference Analysis*: To assess the statistical significance of the differences between the proposed approach and existing structured pruning methods, we performed a rank-based Critical Difference analysis, excluding CFSP since it

is reported only for LLaMA-2 7B. First, the Friedman test was applied to examine whether the compared methods differ significantly. After rejecting the null hypothesis, we performed a post-hoc Wilcoxon signed-rank test with Holm’s correction at $\alpha = 0.05$. The resulting average ranks are visualized using a Critical Difference (CD) diagram, as shown in Figure 6. Figure 6 shows that the proposed approach using the mixed calibration set achieves the best average rank, followed by the proposed approach using WikiText-2 calibration. The better rank achieved with mixed calibration can be attributed to improved pruning stability, as the mixed calibration set exposes the search process to more diverse activation patterns during the search.

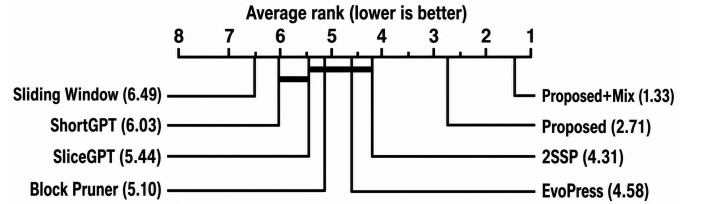


Fig. 6. Critical difference diagram showing the pairwise statistical comparison of structured pruning methods with the proposed approach.

A. Ablation studies

We conducted ablation studies to isolate the main design choices of the proposed framework. We performed ablation studies on LLaMA-2-7B. The ablation studies are conducted at 37.5% sparsity and the proxy-correlation analysis additionally compares 37.5% and 50% sparsity to evaluate how the proxy behavior changes under more aggressive pruning. The analysis focuses on five primary questions: (i) whether the proxy metrics used during the search correlate with downstream accuracy (ii) how the calibration set affects downstream performance and latency (iii) whether the two-stage design is more effective than using either stage alone (iv) which objective combination provides the best accuracy–efficiency trade-off and (v) how different stage-2 sparsity allocation and importance estimation strategies affect the pruned model’s performance.

1) *Proxy Metrics and Downstream Accuracy*: To validate our Stage 1 multi-objective search, we analyze the correlation between the resulting Pareto fronts (optimized for KL divergence and perplexity) and final downstream accuracy. We compute these proxy metrics for all candidate pruned models

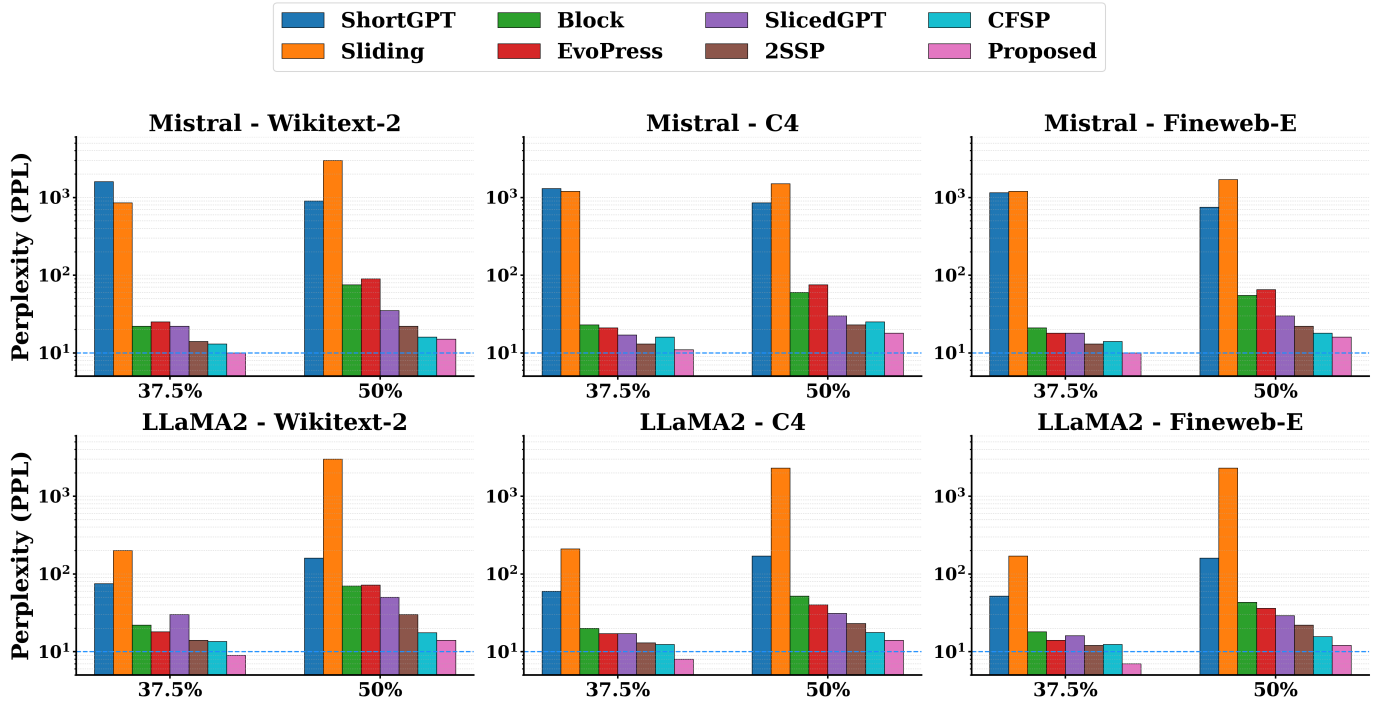


Fig. 5. Perplexity (PPL) comparison across Mistral and LLaMA2 on WikiText-2, C4, and FineWeb-Edu.

and compare them against their zero-shot performance. The motivation behind this analysis is to validate these proxies, as our search procedure relies on computationally inexpensive evaluations to filter candidates before selecting them for expensive downstream validation.

TABLE IV
CORRELATION AND SQUARED-CORRELATION ANALYSIS WITH
DOWNSTREAM AVERAGE ACCURACY.

Sparsity	Pred.	r	r^2	ρ	ρ^2
0.375	KL div.	-0.967	0.935	-0.794	0.630
0.375	PPL	-0.358	0.128	-0.006	0.000
0.5	KL div.	0.417	0.174	0.381	0.145
0.5	PPL	-0.828	0.685	-0.786	0.617

As shown in Figure 8 and Table IV, KL divergence is a stronger proxy for downstream accuracy at moderate sparsity, whereas perplexity becomes more reliable under aggressive pruning. At 37.5% sparsity, KL divergence is strongly correlated with downstream accuracy ($r = -0.967$), but this relationship weakens at 50% sparsity, where perplexity shows a stronger association. This indicates that KL divergence is better suited to conservative pruning, while perplexity is more effective when pruning substantially affects language-modeling performance.

2) *Calibration-Set Size*: The calibration size controls the quality of the activation statistics used for pruning. Figure 7 summarizes the relationship between calibration size, downstream performance, latency, and throughput. It shows that increasing the calibration set from 128 to 2048 samples consistently improves deployment metrics: mean latency decreases from approximately 4.33 s to approximately 4.03 s, while throughput increases from approximately 118.5 to approximately 126.2 tokens/s. Downstream accuracy follows the same trend, rising from approximately 39.7% at 128 samples to approximately 49.8% at 2048 samples.

The largest accuracy gain occurs between 512 and 1024 samples, suggesting that smaller calibration sets are insufficient to capture task-relevant behavior. However, the latency curve begins to saturate after 512 samples, showing that efficiency gains plateau earlier than accuracy gains. This result supports using sufficiently large calibration sets while highlighting the practical trade-off between calibration cost and marginal performance improvement. Given the diminishing returns between 1024 and 2048 samples, we selected a calibration size of 1024 for our experiments to avoid unnecessary computational run-time.

3) *Stage-Wise Combination Ablation*: We evaluate whether a two-stage pruning approach is essential or if single-stage pruning is equally effective. To demonstrate this we have performed multiple experiments with alternative pipeline decompositions: depth-only, width-only, sequential, reversed, and combined configurations as show in Figure 9. The results shows that depth pruning removes blocks while keeping balance between performance preservation and efficiency, whereas width pruning alone offers fine control but misses block-level redundancy. Combining the two allows the method to first identify globally efficient candidates and then refine them through component-

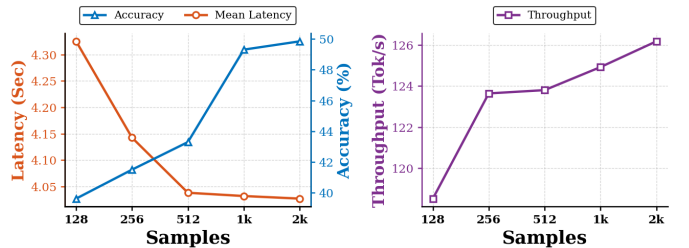


Fig. 7. Effect of calibration sample size on latency, throughput, and downstream accuracy.

level pruning with balance between latency and accuracy. This shows the importance of two-stage design. We also evaluate a reversed stage-wise setting, where width-wise pruning (Stage 2) is applied first, followed by depth pruning (Stage 1). The ratio of sparsity is divided equally in both stages where stage 2 apply 17.5% sparsity rate and stage 1 applies 17.5% sparsity rate. In this setting, the depth-pruning candidates are ranked using both objectives, and the best candidate is selected. As shown in Figure 9, this reversed order gives the lowest downstream accuracy (0.469) and the highest latency (4.08 s), indicating that applying width-wise pruning first removes useful sub-components too early and yields a weaker accuracy-latency trade-off than the proposed H2SP order.

4) *Objective-Combination Analysis*: Our multi-objective formulation consists of two main objectives: one for model performance and one for efficiency. We evaluate two performance criteria (KL-divergence and perplexity) and two efficiency criteria (latency and number of parameters). The results for the different objective combinations are given in Table V.

TABLE V
OBJECTIVE-COMBINATION COMPARISON. LOWER INFERENCE AND PPL ARE BETTER; HIGHER ACCURACY IS BETTER.

1st Objective	2nd Objective	Inference↓	Accuracy↑	PPL↓
Perplexity	Params	7.95	0.5821	8.2943
KL-Divergence	Params	5.86	0.5930	8.1393
Perplexity	Latency	6.60	0.5732	9.2695
KL-Divergence	Latency	4.86	0.5647	7.48

With respect to the first objective, KL-Divergence generally outperforms perplexity in efficiency and language-modeling performance, while the downstream-accuracy comparison depends on the second objective. When the number of parameters is used as the second objective, KL-Divergence yields lower inference time (5.86 s vs. 7.95 s), higher mean downstream accuracy (0.5930 vs. 0.5821, averaged over tasks), and lower average perplexity (8.14 vs. 8.29), computed over WikiText-2, C4, and FineWeb-Edu.

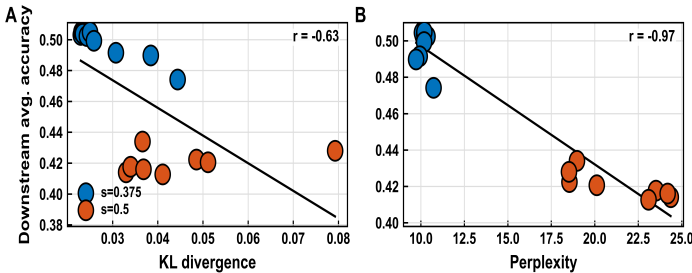


Fig. 8. Correlation between evaluation proxies and downstream accuracy across sparsity levels.

When latency is used as the second objective, the optimization prioritizes runtime efficiency and perplexity over downstream accuracy while still maintaining a balanced trade-off between the objectives. In this setting, KL/Latency achieves the lowest inference time (4.86 s) and the lowest perplexity (7.49), at the cost of a reduced mean downstream accuracy of 0.565. Overall, these results reveal a clear accuracy-latency trade-off: KL/Params is preferable when accuracy preservation is prioritized, whereas KL/Latency is preferable under stringent latency constraints, offering an approximately 17% reduction in

inference time at the cost of a 0.028 absolute (2.8 percentage-point) decrease in mean downstream accuracy.

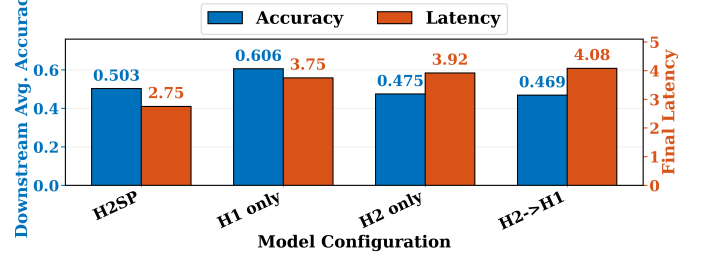


Fig. 9. Downstream accuracy and latency for stage-wise pruning configurations.

5) *Stage-2 Importance Estimator Ablation*: We perform a comparative study of different importance estimation strategies in Stage 2 while keeping Stage 1 fixed. Table VI compares Stage 2 pruning methods in terms of average downstream accuracy, average perplexity over WikiText-2, C4, and FineWeb-Edu, and latency.

TABLE VI
COMPARISON OF STAGE-2 IMPORTANCE ESTIMATORS.

Method	Downstream Avg. ↑	Avg. PPL ↓	Latency ↓
Importance [8]	0.4634	29.9688	3.7887
Wanda-SP [11]	0.4927	21.6250	3.7604
Taylor [1]	0.4821	23.6250	3.7659
Gradient [18]	0.4811	23.5312	3.7610
WActiGrad [9]	0.4413	26.3438	3.9623

Table VI shows that Wanda-SP achieves the best result across all three reported metrics, with the highest downstream average accuracy (0.4927), the lowest average PPL (21.6250), and the lowest latency (3.7604). This indicates that the activation-aware salience metric of Wanda-SP [11] preserves structures that are useful not only for downstream prediction but also for language-modeling perplexity and inference speed in this setting. Taylor and Gradient pruning provide the next-best downstream accuracy and perplexity, with Gradient achieving slightly lower PPL and latency than Taylor, while Taylor gives slightly higher downstream accuracy. In contrast, WActiGrad [9] obtains the lowest downstream accuracy (0.4413) and the highest latency (3.9623), making it the least favorable estimator in this ablation. Overall, these results support using Wanda-SP as the Stage 2 importance estimator for the evaluated configuration.

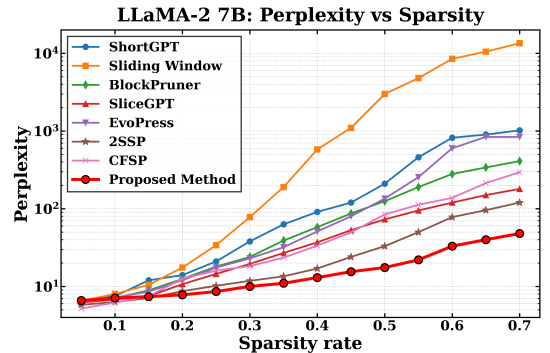


Fig. 10. Comparison of the perplexity values in log scale on different sparsity rates of the proposed approach with existing studies

6) *Stage-2 Sparsity Allocation Methods Ablation*: Table VII compares the Stage 2 sparsity-allocation strategies against the

TABLE VII

STAGE-2 ALLOCATION-METHOD COMPARISON. LOWER PPL AND LATENCY ARE BETTER; HIGHER THROUGHPUT AND DOWNSTREAM AVERAGE ARE BETTER.

Method	Avg. PPL ↓	Latency ↓	Throughput ↑	Downstream Avg. ↑
Uniform	15.8694	3.8216	133.98	0.4572
LSA [19]	13.8296	3.8948	131.60	0.4038
DSA [20]	12.0280	3.8965	131.47	0.4142
DLP [21]	10.3632	3.8641	132.53	0.4898
ER [23]	10.4251	3.9467	129.87	0.4929
OWL [22]	10.3745	3.8689	132.35	0.4898
ER+ [23]	10.3632	3.8885	131.67	0.4898
Proposed	9.6309	3.7871	135.21	0.5142

proposed allocation strategy. The proposed method achieves the best value for every reported metric, with the lowest average PPL (9.6309), lowest latency (3.7871), highest throughput (135.21 tokens/s), and highest downstream average accuracy (0.5142). Among the baselines, ER [23] gives the strongest downstream accuracy (0.4929) but has the highest latency and lowest throughput, while DLP [21], OWL [22], and ER+ provide similar downstream accuracy (0.4898) with lower PPL than Uniform, LSA [19], and DSA [20]. These results indicate that the proposed parallel allocation search improves the accuracy-efficiency trade-off rather than optimizing only a single metric.

Figure 11 shows that the selected LLaMA-2-7B candidate uses non-uniform layer-wise pruning ratios.

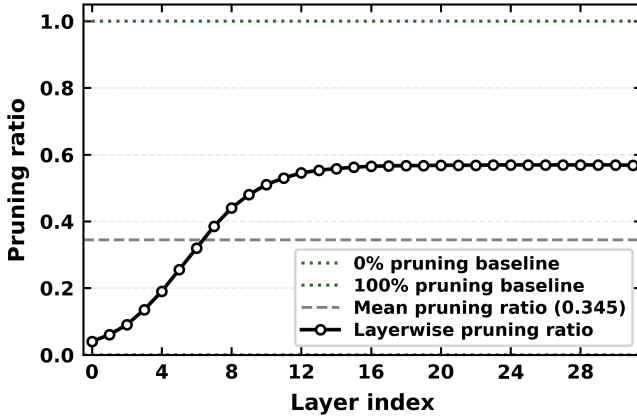


Fig. 11. Layer-wise pruning ratios for the selected LLaMA-2-7B candidate.

VI. CONCLUSION

We presented a hardware-aware two-stage structured pruning framework for deployment-oriented LLM compression. We evaluate the method on Mistral, Qwen, Phi, and LLaMA at 37.5% and 50% sparsity rates. The results show improved average commonsense-reasoning performance over existing pruning and sparsity-allocation methods, competitive or lower perplexity on WikiText-2, C4, and FineWeb-Edu, and reduced 100-token inference latency on both NVIDIA A100 and Jetson edge hardware. Overall, combining Pareto-based coarse pruning with latency-aware Bayesian sparsity allocation provides an effective accuracy-efficiency trade-off for structured LLM compression. Future work will focus on improving search scalability through search-space partitioning and extending evaluation to more edge platforms.

ACKNOWLEDGMENT

This work was partially supported by the CPER project Cornella and the EXAMA (Methods and Algorithms at Exascale) project under grant ANR-22-EXNU-0002.

DECLARATION OF AI-TOOL USAGE

The authors used ChatGPT only to improve grammar and sentence structure. They reviewed and edited the resulting text and take full responsibility for the manuscript.

REFERENCES

- [1] Ma, X., Fang, G. & Wang, X. Llm-pruner: On the structural pruning of large language models. *Advances In Neural Information Processing Systems*. 36 pp. 21702-21720 (2023)
- [2] Gromov, Andrey, et al. "The Unreasonable Ineffectiveness of the Deeper Layers." The Thirteenth International Conference on Learning Representations. (2024)
- [3] Sandri, F., et al. 2ssp: A two-stage framework for structured pruning of llms. *ArXiv Preprint ArXiv:2501.17771*. (2025)
- [4] Ashkboos, Saleh, et al. "SliceGPT: Compress Large Language Models by Deleting Rows and Columns." The Twelfth International Conference on Learning Representations. (2024)
- [5] Zhong, L., et al. Blockpruner: Fine-grained pruning for large language models. *Findings Of The Association For Computational Linguistics: ACL 2025*. pp. 5065-5080 (2025)
- [6] Men, X., et al. Shortgpt: Layers in large language models are more redundant than you expect. *Findings Of The Association For Computational Linguistics: ACL 2025*. pp. 20192-20204 (2025)
- [7] Ding, X., et al. A Sliding Layer Merging Method for Efficient Depth-Wise Pruning in LLMs. *ArXiv Preprint ArXiv:2502.19159*. (2025)
- [8] Sreenivas, Sharath Turuvekere, et al. "LLM Pruning and Distillation in Practice: The Minitron Approach." *arXiv preprint arXiv:2408.11796*. (2024)
- [9] Chitty-Venkata, Krishna Teja, et al. "WActiGrad: structured pruning for efficient finetuning and inference of large language models on AI accelerators." European Conference on Parallel Processing. Cham: Springer Nature Switzerland, 2024.
- [10] Sieberling, O. et al. EvoPress: Accurate Dynamic Model Compression via Evolutionary Search. *Forty-second International Conference On Machine Learning*. (2025).
- [11] Sun, M., Liu, Z., Bair, A. & Kolter, J. Z. A Simple and Effective Pruning Approach for Large Language Models. *International Conference on Learning Representations*. (2024).
- [12] Touvron, Hugo, et al. "Llama: Open and efficient foundation language models." *arXiv preprint arXiv:2302.13971* (2023).
- [13] Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., et al. Mistral 7B. *ArXiv preprint arXiv:2310.06825* (2023).
- [14] Deb, Kalyanmoy, et al. "A fast and elitist multiobjective genetic algorithm: NSGA-II." *IEEE transactions on evolutionary computation* 6.2 (2002)
- [15] Wang, Y., Ma, M., Wang, Z., Chen, J., Shan, L., Yang, Q., Xu, D., Liu, M. & Qin, B. CFSP: An Efficient Structured Pruning Framework for LLMs with Coarse-to-Fine Activation Information. *Proceedings Of The 31st International Conference On Computational Linguistics*. pp. 9311-9328 (2025).
- [16] Ahmed, Imtiaz, et al. "Qwen 2.5: A comprehensive review of the leading resource-efficient LLM with potential to surpass all competitors." *Authorea Preprints* (2025).
- [17] Abdin, M., et al. *Phi-4 Technical Report*. *arXiv preprint arXiv:2412.08905* (2024).
- [18] Huang, Wei, Anda Cheng, and Yinggui Wang. "GradPruner: Gradient-Guided Layer Pruning Enabling Efficient Fine-Tuning and Inference for LLMs." *arXiv preprint arXiv:2601.19503* (2026).
- [19] Yang, Zhiguo, Changjian Deng, Qinke Chen, Zijing Zhou, and Jian Cheng. "LSA: Layer-wise Sparsity Allocation for Large Language Model Pruning Based on Minimal Linear Reconstruction Error." *International Conference on Learning Representations (ICLR)*, 2026.
- [20] Li, Lujun, Peijie Dong, Zhenheng Tang, Xiang Liu, Qiang Wang, Wenhan Luo, Wei Xue, Qifeng Liu, Xiaowen Chu, and Yike Guo. "Discovering Sparsity Allocation for Layer-wise Pruning of Large Language Models." *Advances in Neural Information Processing Systems* 37 (NeurIPS), 2024.
- [21] Chen, Yuli, Bo Cheng, Jiale Han, Yingying Zhang, Yingting Li, and Shuhao Zhang. "DLP: Dynamic Layerwise Pruning in Large Language

- Models.” Proceedings of the 42nd International Conference on Machine Learning, PMLR 267:7934–7956, 2025.
- [22] Yin, Lu, You Wu, Zhenyu Zhang, Cheng-Yu Hsieh, Yaqing Wang, Yiling Jia, Gen Li, Ajay Kumar Jaiswal, Mykola Pechenizkiy, Yi Liang, Michael Bendersky, Zhangyang Wang, and Shiwei Liu. ”Outlier Weighed Layerwise Sparsity (OWL): A Missing Secret Sauce for Pruning LLMs to High Sparsity.” Proceedings of the 41st International Conference on Machine Learning, PMLR 235:57101–57115, 2024.
 - [23] Huang, Weizhong, Yuxin Zhang, Xiawu Zheng, Fei Chao, and Rongrong Ji. ”Determining Layer-wise Sparsity for Large Language Models Through a Theoretical Perspective.” Proceedings of the 42nd International Conference on Machine Learning, PMLR 267:26175–26201, 2025.